ShowTracker — Redis Practicum

1. Problem Requirements and Conceptual Model

ShowTracker allows users to create, browse, edit, and delete shows (concerts/events) and venues. The core database is SQLite, with two tables:

Venue — venue_id (PK), name, city, state, capacity, genreTags
Show — show_id (PK), date, ticketPrice, genre, venue_id (FK → Venue)

While SQLite handles persistent CRUD operations well, certain access patterns benefit from the speed of an in-memory store. Redis is introduced to handle two high-frequency features:

1. Most Viewed Venues (Sorted Set) — Every time a user views a venue's show listing, a counter is incremented. This powers a "Trending Venues" leaderboard. Because view counts update on every page load and are queried frequently, Redis sorted sets are ideal: O(log N) writes and O(log N + M) ranked reads.

1.1 Functional Requirements

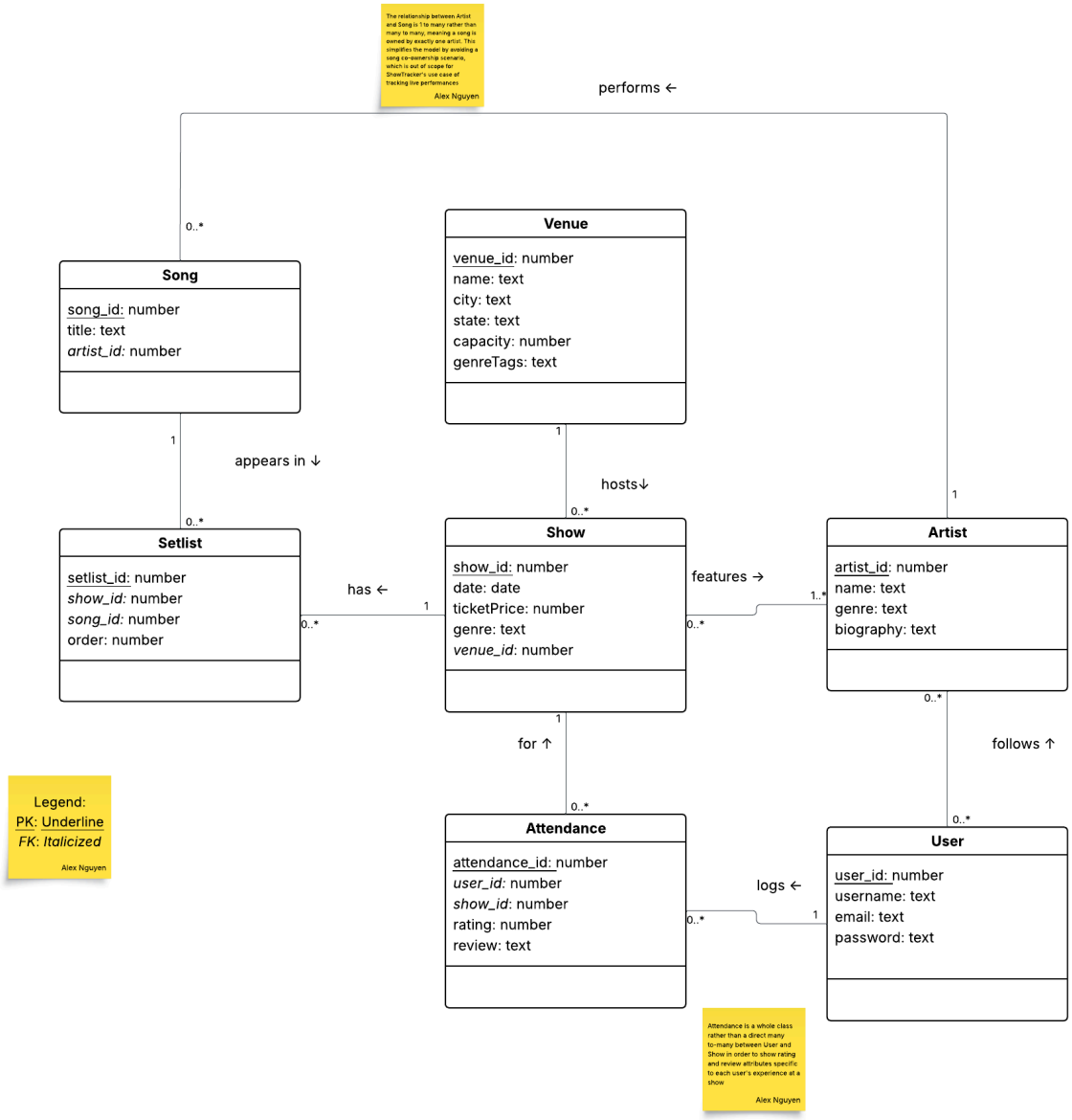
- Users can browse venues; each visit increments a per-venue view counter in Redis. • Users can retrieve a ranked leaderboard of most-viewed venues.
- Users can remove a venue from the trending leaderboard.
- Users can reset (clear) the entire leaderboard.
- All Redis data supplements the primary SQLite store — it is not the source of truth.

1.2 Non-Functional Requirements

- Read latency for trending-venues queries should be under 5 ms.
- The system should gracefully degrade: if Redis is unavailable, the app still serves data from SQLite.
- Data in Redis is ephemeral — it can be rebuilt from application event logs.

Conceptual Model

The diagram below shows the existing ShowTracker entities. The full conceptual model includes Venue, Show, Artist, Song, Setlist, User, and Attendance. The current SQLite implementation covers Venue and Show. The Redis layer adds a sorted set for tracking trending venues, tied to the Venue entity. This conceptual model is reused from project 1.



3. Redis Data Structure Design

3.1 Sorted Set — Trending Venues

Key: `trendingVenues`
Members: Venue IDs from SQLite (e.g., `2`)
Scores: Number of times the venue listing has been viewed
Purpose: Enables O(log N) increment on each page view and O(log N + M) retrieval of

the top X most viewed venues, sorted by view count descending.

Example state:

Member (venue_id)	Score (Views)
2 (Little Field — Oakland, CA)	47
5 (The Fillmore — San Francisco, CA)	31
8 (Madison Square Garden — New York, NY)	22

4. Redis CRUD Commands

4.1 Sorted Set — Trending Venues (Full CRUD)

Create — Record a venue view:

```
ZINCRBY trendingVenues 1 2
# Increments venue_id 2's view count by 1. Creates the key/member if they
don't exist.
```

Read — Get trending venues:

```
ZREVRANGE trendingVenues 0 -1 WITHSCORES
# Returns ALL venues sorted by views descending.

ZREVRANGE trendingVenues 0 4 WITHSCORES
# Returns top 5 most viewed venues.

ZSCORE trendingVenues 2
# Returns the view count for venue_id 2.

ZCARD trendingVenues
# Returns how many venues have been viewed.
```

Update — Adjust a view count:

```
ZINCRBY trendingVenues 1 2
# Increment by 1 on each page view (same command as create — idempotent upsert).

ZADD trendingVenues 50 2
# Override the score to an exact value (e.g. admin
correction).
```

Delete — Remove a venue or clear all:

```
ZREM trendingVenues 2
# Remove venue_id 2 from the sorted set.
```

```
DEL trendingVenues
# Delete the entire sorted set.
```

4.2 Initialization / Reset

```
FLUSHALL
# Clear all Redis data (development only).

# Seed sample data:
ZINCRBY trendingVenues 47 2
ZINCRBY trendingVenues 31 5
ZINCRBY trendingVenues 22 8
```

5. CRUD Summary

Operation	Sorted Set (Trending Venues)
Create	ZINCRBY / ZADD
Read	ZREVRANGE, ZSCORE, ZCARD
Update	ZINCRBY / ZADD (overwrite)
Delete	ZREM / DEL